

SMS Spam Detection Active Learning Pipeline

Project Documentation

Branch: SPAM_DETECTION_V4

Generated: 27 February 2026, 06:51 MYT

Repository: github.com/uc-cjkhoh/Spam_Detection

Table of Contents

1. Project Overview	3
2. Project Objective	3
3. Installation	4
4. Project Detail / Program Procedure	5
5. Project Architecture	7

1 Project Overview

The SMS Spam Detection project is an **active-learning machine learning pipeline** designed to classify incoming SMS messages as spam or ham (legitimate). The system combines sentence embeddings, a FAISS vector store, LightGBM classification, and incremental human-in-the-loop labelling to continuously improve detection accuracy over time.

Key Highlights

- Model: LightGBM Classifier with class-weight balancing
- Embeddings: sentence-transformers/all-MiniLM-L6-v2 (384-dim)
- Vector Store: FAISS-backed similarity search
- Experiment Tracking: MLflow
- Pipeline Orchestration: Prefect
- Deployment: FastAPI containerised with Docker

Technology Stack

Component	Technology	Purpose
ML Model	LightGBM	Binary spam classification
Embeddings	HuggingFace / MiniLM-L6	Semantic text representation
Vector Store	FAISS	Similarity search & pseudo-labelling
Orchestration	Prefect	Pipeline task management & logging
Tracking	MLflow	Experiment and model registry
Database	MySQL	Store labelled data and results
API	FastAPI + Uvicorn	Real-time classification endpoint
Containerisation	Docker	Portable deployment
Balancing	SMOTE (imbalanced-learn)	Handle class imbalance

2 Project Objective

The primary goal is to build a **production-grade, continuously improving SMS spam filter** that can handle real-world Malaysian SMS patterns including obfuscated text, mixed scripts, and evolving spam tactics — with minimal manual labelling effort over time.

Primary	Accurately classify SMS messages as spam (1) or ham (0) in real time via API.
Secondary	Minimise human labelling effort through active learning and pseudo-labelling via FAISS similarity search.
Quality	Maintain high precision (>97%) to avoid false positives while achieving strong recall (>89%) for spam detection.
Scalability	Support batch processing and horizontal scaling through containerised FastAPI deployment.

Observability

Track all experiments, model versions, and evaluation metrics through MLflow for auditability.

3.1 Prerequisites

Python	3.10+
MySQL	8.0+ (accessible at 10.168.51.196:3306)
MLflow Server	Running at http://10.168.49.12:5000
Prefect Server	Running on configured host (see .env PREFECT_API_URL)
Docker	Optional — required for containerised deployment

3.2 Setup Steps

Step 1 — Clone Repository

```
git clone https://github.com/uc-cjkhoh/Spam_Detection
cd Spam_Detection
```

Step 2 — Create Virtual Environment

```
python -m venv .venv
source .venv/bin/activate # Linux / macOS
.venv\Scripts\activate # Windows
```

Step 3 — Install Dependencies

```
pip install -r requirements.txt
```

Step 4 — Configure Environment

Create a .env file in the project root with the following content:

```
PREFECT_HOME=./prefect
PREFECT_LOCAL_STORAGE_PATH=./prefect/storage
PREFECT_MEMO_STORE_PATH=./prefect/memo_store.toml
PREFECT_API_URL=http://10.168.49.12:4200/api
PREFECT_LOGGING_SETTINGS_PATH=./prefect/logging.yml
PREFECT_RESULTS_PERSIST_BY_DEFAULT=True
```

Step 5 — Configure config.yaml

Edit configs/config.yaml with the following key settings:
mlflow_uri, database credentials, embedding model, experiment_name

Step 6 — Start Infrastructure

```
prefect server start --host 0.0.0.0
mlflow server --host 0.0.0.0
```

Step 7 — Run Pipeline

```
python main.py
```

3.3 API Deployment

To deploy the FastAPI inference endpoint:

```
cd ./deploy
pip install -r requirements.txt
uvicorn main:app --host 10.168.49.12 --port 7654 --reload
```

Access Swagger UI at: <http://10.168.49.12:7654/docs>

Docker Deployment

Docker deployment is **not available yet** and is pending future implementation. The Dockerfile is included in `deploy/` for reference only.

The pipeline is orchestrated by Prefect and executed via **main.py**. It follows a structured flow of setup, data loading, training, evaluation, and optional active-learning cycles.

4.1 Pipeline Flow (Prefect Tasks)

#	Task	Description
1	setup_environment	Connects to MLflow and MySQL, initialises HuggingFace embedding model, LGBM model, and FAISS vector store. Loads existing index from disk if available; otherwise builds from initial_data query.
2	load_training_data	Retrieves all embeddings and labels from FAISS. Performs feature engineering and z-score normalisation. Merges with human-labelled data from MySQL. Applies SMOTE oversampling to balance classes.
3	load_testing_data	Queries test split from MySQL (day=6 from new_initial_data, day=22 from initial_data). Applies same feature engineering and scaler transform. Generates embeddings and stacks with engineered features.
4	model.fit	Trains LGBMClassifier on the combined feature + embedding matrix (x_train, y_train). Class weights are balanced automatically within LightGBM.
5	model.evaluate	Starts an MLflow run. Logs model parameters, registers model to MLflow Model Registry. Runs mlflow.models.evaluate() to capture accuracy, precision, recall, F1, ROC-AUC, and log-loss.
6	Active Learning*	Currently commented out. Planned: stratified sampling of new SMS batches, confidence-based pseudo-labelling, uncertain sample extraction for human review, results saved to MySQL label_by_vectoradb table.

* Active learning module is currently commented out pending automation of labelling workflow.

4.2 Text Preprocessing & Feature Engineering

The **SMSTextCleaner** class in *src/data_loader/preprocessing.py* performs two roles: text normalisation before embedding, and feature extraction for the LightGBM input.

Feature	Description
emoji_count	Number of emoji characters in the message
has_phone / has_currency / has_url	Binary flags for contact/financial/link indicators
uppercase_ratio	Ratio of uppercase characters — spam often uses ALL CAPS
digit_ratio	Fraction of digits — phone numbers inflate this
special_char_ratio	Ratio of non-alphanumeric characters
length	Total character count of message
char_substitution_count	Leet-speak substitutions (0->o, 3->>e) used to evade filters

financial_keyword_count	Matches: loan, free, bonus, prize, cash, etc.
action_keyword_count	Matches: reply, whatsapp, click, register, etc.
urgency_keyword_count	Matches: urgent, now, limited, hurry, etc.
suspicious_keyword_count	Matches: AKPK, CTOS, CCRIS, blacklisted, etc.
has_fragmented_phone	Detects spaced-out phone numbers (0 1 1 6 0 5 ...)
call_to_action_count	Count of CTA phrases found in message
has_unicode_special	Detects bold/italic/fancy Unicode fonts
contact_method_diversity	Count of different contact channels mentioned

4.3 Model Inference (API Endpoint)

The FastAPI app in *deploy/main.py* exposes a **POST /classify** endpoint. On each request it: (1) extracts hand-crafted features, (2) generates sentence embeddings, (3) concatenates both into the feature matrix, (4) runs LightGBM predict and predict_proba, and (5) asynchronously saves the result to MySQL *api_result_2* table.

5.1 System Architecture Overview

The system is composed of four logical layers: **Data Layer**, **ML Pipeline Layer**, **Inference Layer**, and **Monitoring Layer**.

[MySQL Database]	sms_spam_cd schema initial_data new_initial_data label_by_vectordb metadata_result data_tdr_spam_filter
[ML Pipeline (Prefect)]	main.py → setup_environment → load_training_data → load_testing_data → model.fit → model.evaluate
[Active Learning Loop]	Stratified Sampling → Feature Extraction → Embedding → Confidence Scoring → Human Review → Relabel → Retrain
[Vector Store (FAISS)]	sms_embeddings_*.pkl similarity_search_by_vector pseudo-label uncertain samples
[Inference API (FastAPI)]	POST /classify → Feature Engineering + Embedding → LightGBM predict → Save to MySQL
[Monitoring (MLflow)]	Experiment tracking Model Registry Metric logging Model versioning

5.2 Repository Structure

Path	Description
main.py	Entry point — Prefect flow definition
configs/config.yaml	All runtime configurations
src/ml/model_training.py	LGBM model class with fit/predict/evaluate
src/data_loader/preprocessing.py	SMSTextCleaner + feature_engineering task
src/data_loader/database.py	MySQL connection and query execution
src/vector_database/vectorstore.py	FAISS wrapper with similarity labelling
src/config_folder/config_loader.py	Loads and parses config.yaml
src/utis/util.py	SMOTE oversampling, helper functions
data_validation/	Pydantic validators for config, model, vectorstore
deploy/main.py	FastAPI app — /classify endpoint
deploy/app/preprocessing.py	Shared feature engineering for API
deploy/app/schema.py	Pydantic request/response schemas
deploy/Dockerfile	Docker image definition

simulate_request.py	Load testing / API simulation script
.github/workflows/	CI: unit_test_workflow.yaml (GitHub Actions)
setup_initial_data.py	Script to bootstrap initial training data

5.3 Data Flow Diagram

Source	→	Process	→	Destination
MySQL (initial_data)	→	FAISS vectorstore init	→	sms_embeddings_*.pkl
FAISS + MySQL (labeled)	→	Feature Eng. + SMOTE	→	x_train / y_train
MySQL (test split)	→	Feature Eng. + Embeddings	→	x_test / y_test
x_train, y_train	→	LightGBM.fit()	→	Trained Model
Trained Model + x_test	→	MLflow.models.evaluate()	→	Metrics + Registry
POST /classify payload	→	Feature Eng. + Embed + Predict	→	JSON result + MySQL

Experiment: initial_data+new_initial_data_V2 | **Model:** LightGBM (lightgbm_model) | **Test Set:** 4,662 samples

6.1 Core Metrics Summary

Metric	Value	Metric	Value
Accuracy	97.15%	ROC-AUC	99.74%
Precision	98.28%	PR-AUC	99.30%
Recall	89.94%	Log Loss	0.0676
F1 Score	93.92%	Test Samples	4,662

6.2 Confusion Matrix

	Predicted Ham (0)	Predicted Spam (1)
Actual Ham (0)	3,501 True Negatives	18 False Positives
Actual Spam (1)	115 False Negatives	1,028 True Positives

6.3 Detailed Analysis

Precision (98.28%) — Excellent	Of all messages the model classified as spam, 98.28% were genuinely spam. Only 18 legitimate messages (ham) were falsely flagged across 4,662 test samples. This minimises user frustration from false spam alerts.
Recall (89.94%) — Good	115 spam messages were missed (false negatives). While strong, this is the main area for improvement. The active learning loop is designed to iteratively capture these edge cases through human review and re-labelling of uncertain predictions.
F1 Score (93.92%) — Strong Balance	The harmonic mean of precision and recall shows a well-balanced model. The slight lean towards precision over recall is intentional — false positives (blocking legitimate messages) are more disruptive in production than missing some spam.
ROC-AUC (99.74%) — Near-Perfect Discrimination	The model has near-perfect ability to distinguish spam from ham across all probability thresholds. This confirms robust generalisation beyond the default 0.5 decision boundary.

PR-AUC (99.30%) — Excellent on Imbalanced Data	Precision-Recall AUC is the most reliable metric for imbalanced datasets. 99.30% confirms the model maintains both high precision and high recall even when operating on the minority class (spam), validated against the SMOTE-oversampled training distribution.
Log Loss (0.0676) — Well-Calibrated	A low log-loss indicates the model's probability outputs are reliable and well-calibrated. This is critical for the confidence-score-based active learning threshold (confidence_score_threshold: 0.98).

6.4 Recommendations

Action	Priority	Detail
Improve Recall	MEDIUM	Experiment with lowering the LightGBM classification threshold from 0.5 to ~0.4 to capture more spam at a marginal precision cost. Monitor via MLflow.
Activate Active Learning	HIGH	Uncomment and deploy the active learning loop. Human review of ~500 uncertain samples per cycle should progressively reduce false negatives.
Initial Data Reduction	LOW	Implement HDBSCAN clustering on 5% random samples to identify representative patterns, reducing initial labelling burden while preserving coverage.
Threshold Tuning	MEDIUM	confidence_score_threshold is currently 0.98. Evaluate whether reducing to 0.95 would auto-label more samples without sacrificing label quality.